SIMATS ENGINEERING

ASSESSMENT TOOL 1: Algorithm Design Challenge

COURSE NAME: ARTIFICIAL INTELLIGENCE AND EXPERT SYSTEMS **COURSE CODE: MLA0101**

COURSE OUTCOME COVERED

CO1: *Apply search algorithms and heuristic techniques to solve state-space search and optimization problems.* (BTL 3)

Assessment Tool Weightage: 40 %

Total Marks: 100 **Time: 90 Mins**

No. of Questions: 5

Annexure A

Algorithm Design Challenge

Code of Conduct:

*I **M. Veera Naga Sai(192425253)**(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: **M. Veera Naga Sai**

Criterion	Weightage	Marks Obtained
Problem Analysis and Understanding	15%	
AI Algorithm Selection & Justification	15%	
Algorithm Design / Pseudocode	25%	
Application of AI Concepts (Greedy, A*, CSP, Minimax, etc.)	15%	
Diagram / State Space / Search Tree Representation	10%	
Complexity Analysis and Solution Efficiency	10%	
Originality, Critical Thinking, and Presentation	10%	
Total	100%	

Signature of the Faculty Total Marks Awarded

Answer all the Questions

Q.No	Question	Topics
1	**Design an algorithm using Greedy Best-First Search to find a path from a source node to a goal node for the given graph. Clearly define the heuristic function, show the node expansion order, and justify why Greedy Best-First Search is suitable for this problem.**	Informed Search, Greedy Best-First Search, Heuristic Functions
2	**A delivery robot must travel from a warehouse to a customer through a weighted road network. Design an A Search algorithm to determine the optimal path. Calculate g(n), h(n), and f(n) for each expanded node, and explain why A guarantees the optimal solution.**	A* Search, Heuristic Functions
3	**A university needs to prepare an examination timetable without scheduling conflicting subjects at the same time. Design the problem as a Constraint Satisfaction Problem (CSP) by identifying variables, domains, and constraints. Explain how MRV and Forward Checking improve the efficiency of the solution.**	CSP, MRV, Forward Checking
4	**Design the Minimax algorithm for the given game tree to determine the best move for the MAX player. Then apply Alpha–Beta Pruning to the same tree and indicate which nodes are pruned. Compare the number of nodes evaluated before and after pruning.**	Game Playing, Minimax, Alpha–Beta Pruning
5	**A warehouse robot operates in a dynamic environment where obstacles may appear unexpectedly. Design an intelligent search strategy using either Local Search or an Online Search Agent. Explain how the algorithm adapts to environmental changes and compare its performance with informed search algorithms.**	Local Search, Optimization, Online Search Agents

Structure of the Answer – Algorithm Design Challenge

S.No	Sections	Expected Content
1	**Problem Statement**	Briefly describe the given problem in your own words.
2	**Problem Analysis**	Identify the initial state, goal state, assumptions, and constraints.
3	**AI Technique Selection**	Specify the most appropriate AI algorithm/search technique (e.g., Greedy Best-First Search, A*, CSP, Minimax, Local Search).
4	**Justification of Algorithm**	Explain why the selected algorithm is suitable compared to other search techniques.
5	**State Space Representation**	Represent the problem using a state-space diagram, search graph, game tree, or constraint graph, as applicable.
6	**Variables, Domains, and Constraints (Applicable for CSP problems)**	Identify variables, possible domains, and constraints. If not applicable, mention "Not Applicable".
7	**Heuristic Function (Applicable for Informed Search)**	Define the heuristic function h(n) and explain whether it is admissible or appropriate for the problem. If not applicable, mention "Not Applicable".
8	**Algorithm Design / Pseudocode**	Write the step-by-step algorithm or pseudocode for solving the problem.
9	**Flowchart / Algorithm Diagram**	Draw a flowchart, search tree, game tree, or algorithm workflow illustrating the solution.
10	**Working / Execution Trace**	Demonstrate the execution of the algorithm with at least one example, showing intermediate steps such as node expansion, constraint propagation, or game-tree evaluation.
11	**Complexity Analysis**	Analyze the Time Complexity and Space Complexity of the proposed algorithm.
12	**Advantages and Limitations**	Discuss the strengths and limitations of the selected algorithm for the given problem.
13	**Conclusion**	Summarize the effectiveness of the designed algorithm and suggest possible improvements or alternative approaches.

QUESTION 1 — Greedy Best-First Search

1. Problem Statement

Design an algorithm using Greedy Best-First Search to find a path from a source node S to a goal node G in a given graph. The algorithm should use a heuristic function to select the node that appears closest to the goal.

Consider the following graph:

S → A → C → G

and another possible path:

S → B → C → G

2. Problem Analysis

Initial State: S

Goal State: G

Assumptions:

- Each node has a heuristic value.
- A smaller heuristic value indicates that the node is estimated to be closer to the goal.
- The graph is connected.
- Visited nodes are not expanded repeatedly.

Constraints:

- The search should reach the goal node.
- At every step, the node with the smallest heuristic value is selected.

3. AI Technique Selection

The appropriate AI technique is **Greedy Best-First Search**.

Greedy Best-First Search is an informed search algorithm that uses heuristic information to decide which node should be expanded next.

The evaluation function is:

f(n) = h(n)

4. Justification of Algorithm

Greedy Best-First Search is suitable because it uses heuristic information to move toward the goal quickly.

It is better than uninformed search techniques such as BFS and DFS when useful heuristic information is available.

It is generally faster when the heuristic is good because it focuses on nodes that appear closer to the goal.

However, Greedy Best-First Search does not always produce the shortest path because it considers only h(n) and ignores the cost already travelled.

5. State Space Representation

The state-space representation can be shown as:

S

↓

A (h = 4)

↓

C (h = 2)

↓

G (h = 0)

There is also another branch:

S → B (h = 5) → C → G

Since A has a smaller heuristic value than B, A is selected first.

6. Variables, Domains, and Constraints

Not Applicable.

This problem is an informed search problem and not a Constraint Satisfaction Problem.

7. Heuristic Function

The heuristic function is:

h(n) = estimated cost or distance from node n to the goal

8. Algorithm Design / Pseudocode

1. Start from the source node S.
2. Place S in the OPEN list.
3. Select the node having the smallest heuristic value.
4. If the selected node is the goal G, stop.
5. Otherwise, expand the selected node.
6. Add its unvisited neighboring nodes to the OPEN list.
7. Again select the node with the smallest heuristic value.
8. Continue until the goal is reached or no nodes remain.

9. Flowchart / Algorithm Diagram

START

↓

Place S in OPEN list

↓

Select node with minimum h(n)

↓

Is the node G?

→ **Yes:** Return path and stop.

→ **No:** Expand the node.

↓

Add unvisited successors

↓

Select minimum h(n) node again

↓

Repeat until G is reached

10. Working / Execution Trace

Initially:

S is selected.

The successors are:

- A with h(A) = 4
- B with h(B) = 5

Since:

4 < 5

A is selected.

Next, A leads to C.

C has:

h(C) = 2

Therefore, C is selected.

C leads to G.

G has:

h(G) = 0

Therefore, G is selected and the search terminates.

Node Expansion Order

S → A → C → G

Final Path

S → A → C → G

11. Complexity Analysis

Let:

- b = branching factor
- m = maximum depth

Time Complexity: O(b^m)

Space Complexity: O(b^m)

The actual performance depends on the quality of the heuristic function.

12. Advantages and Limitations

Advantages

- Uses heuristic information.
- Usually reaches the goal quickly.
- Simple to understand and implement.
- Useful when speed is more important than guaranteed optimality.

Limitations

- Does not guarantee the shortest path.
- Performance depends on heuristic quality.
- A poor heuristic can lead to inefficient searching.
- Can consume considerable memory.

13. Conclusion

Greedy Best-First Search is suitable for problems where the main objective is to reach the goal quickly using heuristic information.

For the given example, the algorithm finds:

S → A → C → G

The major principle is:

Greedy Best-First Search → f(n) = h(n)

QUESTION 2 — A* Search

1. Problem Statement

A delivery robot must travel from a warehouse to a customer through a weighted road network.

The objective is to find the path having the minimum total travel cost.

Consider the following weighted graph:

S → A = 2

S → B = 4

A → C = 2

B → C = 1

C → G = 3

2. Problem Analysis

Initial State: S, representing the warehouse.

Goal State: G, representing the customer.

Assumptions:

- Every road has a non-negative cost.
- The heuristic estimates the remaining cost to the goal.
- The road network is known.
- The robot wants the minimum-cost route.

Constraints:

- The total travel cost should be minimized.
- The robot must eventually reach G.
- The heuristic should preferably be admissible.

3. AI Technique Selection

The selected algorithm is *A Search**.

A* combines the actual cost travelled and the estimated remaining cost.

The evaluation function is:

f(n) = g(n) + h(n)

where:

- g(n) = actual cost from S to n
- h(n) = estimated cost from n to G
- f(n) = total estimated cost

4. Justification of Algorithm

A* is suitable because this problem involves:

- Weighted roads.
- A specific destination.
- A requirement for the optimal path.
- Available heuristic information.

A* is better than Greedy Best-First Search because it considers both the path already travelled and the estimated remaining distance.

If the heuristic is admissible, A* guarantees an optimal solution.

5. State Space Representation

The weighted paths are:

S —2→ A —2→ C —3→ G

and

S —4→ B —1→ C —3→ G

The first route has total cost:

2 + 2 + 3 = **7**

The second route has total cost:

4 + 1 + 3 = **8**

Therefore, the first route is optimal.

6. Variables, Domains, and Constraints

Not Applicable.

This is an A* search problem rather than a CSP.

7. Heuristic Function

The heuristic function estimates the remaining cost from a node to the goal.

A* uses:

f(n) = g(n) + h(n)

For example, for A:

g(A) = 2

h(A) = 4

Therefore:

f(A) = 2 + 4 = **6**

An admissible heuristic never overestimates the actual minimum remaining cost.

8. Algorithm Design / Pseudocode

1. Start from S.
2. Set g(S) = 0.
3. Calculate f(S) = g(S) + h(S).
4. Select the node having the smallest f(n).
5. If the selected node is G, return the path.
6. Otherwise, expand the node.
7. Calculate g(n), h(n), and f(n) for its successors.
8. Add the successors to the OPEN list.
9. Select the node with the smallest f(n).
10. Continue until G is reached.

9. Flowchart / Algorithm Diagram

START

↓

Place S in OPEN

↓

Calculate g(n), h(n), and f(n)

↓

Select node with minimum f(n)

↓

Is node G?

→ **Yes:** Return optimal path.

→ **No:** Expand node.

↓

Calculate values for successors

↓

Update OPEN list

↓

Repeat

10. Working / Execution Trace

For S:

g(S) = 0

h(S) = 6

f(S) = 6

After expanding S:

For A:

g(A) = 2

h(A) = 4

f(A) = 6

For B:

g(B) = 4

h(B) = 4

f(B) = 8

A is selected because it has the smaller f-value.

For C:

g(C) = 2 + 2 = 4

h(C) = 2

f(C) = 6

For G:

g(G) = 4 + 3 = 7

h(G) = 0

f(G) = 7

11. Complexity Analysis

Let:

- b = branching factor
- d = depth of optimal solution

Time Complexity: O(b^d)

Space Complexity: O(b^d)

The quality of the heuristic strongly affects the practical performance.

12. Advantages and Limitations

Advantages

- Finds an optimal path when the heuristic is admissible.

- Considers actual and estimated cost.
- Useful for route planning.
- More reliable than Greedy Best-First Search for optimal paths.

Limitations

- Can require large memory.
- A poor heuristic can make it slow.
- More computationally expensive than simple greedy search.

13. Conclusion

A* Search is highly suitable for the delivery robot because it considers both the actual distance travelled and the estimated remaining distance.

The optimal path is:

S → A → C → G

with total cost:

7

The main formula is:

*A → f(n) = g(n) + h(n)**

QUESTION 3 — Examination Timetable Using CSP

1. Problem Statement

A university needs to prepare an examination timetable so that subjects having common students are not scheduled at the same time.

This problem can be formulated as a **Constraint Satisfaction Problem (CSP)**.

Suppose the subjects are:

- A = Mathematics
- B = Physics
- C = Computer Science
- D = Chemistry
- E = English

2. Problem Analysis

Initial State:

No subject has been assigned an examination slot.

Goal State:

Every subject must be assigned a valid examination slot without violating any conflict.

Available slots:

- Slot 1 = Monday Morning
- Slot 2 = Monday Afternoon
- Slot 3 = Tuesday Morning

Subject conflicts:

- Mathematics conflicts with Physics.
- Mathematics conflicts with Computer Science.
- Physics conflicts with Chemistry.
- Computer Science conflicts with Chemistry.
- Computer Science conflicts with English.

Therefore, conflicting subjects cannot have the same slot.

3. AI Technique Selection

The appropriate technique is **Constraint Satisfaction Problem (CSP)**.

CSP represents a problem using:

Variables + Domains + Constraints

For improving efficiency, we use:

- **MRV — Minimum Remaining Values**
- **Forward Checking**

4. Justification of Algorithm

CSP is suitable because examination scheduling naturally contains variables, possible values, and restrictions.

For example:

- Subjects are variables.
- Examination slots are domains.
- Subject conflicts are constraints.

MRV selects the subject with the fewest remaining possible slots.

Forward Checking removes invalid slots from related subjects

5. State Space Representation

The constraint graph is:

A — B

A — C

B — D

C — D

C — E

An edge between two subjects indicates that they cannot be scheduled at the same time.

6. Variables, Domains, and Constraints

Variables

A, B, C, D, E

Domains

Initially:

Variable	Domain
A	{1, 2, 3}
B	{1, 2, 3}
C	{1, 2, 3}
D	{1, 2, 3}
E	{1, 2, 3}

Constraints

$A \neq B$

$A \neq C$

$B \neq D$

$C \neq D$

$C \neq E$

7. Heuristic Function

A traditional heuristic function is:

Not Applicable.

However, MRV acts as a variable-selection strategy.

MRV selects the variable having the smallest number of remaining legal values.

8. Algorithm Design / Pseudocode

1. Check whether all subjects have been assigned.
2. Select an unassigned subject using MRV.
3. Select one available examination slot.
4. Check whether the assignment violates any constraint.
5. If valid, assign the slot.
6. Apply Forward Checking to neighboring subjects.
7. If a neighboring subject has no available slot, backtrack.
8. Continue until all subjects are assigned.
9. Return the completed timetable.

9. Flowchart / Algorithm Diagram

START

↓

Select subject using MRV

↓

Select available slot

↓

Check constraints

↓

Is assignment valid?

→ **No:** Try another slot.

→ **Yes:** Assign slot.

↓

Apply Forward Checking

↓

Any empty domain?

→ **Yes:** Backtrack.

→ **No:** Continue.

↓

All subjects assigned?

→ **Yes:** Timetable completed.

→ **No:** Repeat.

10. Working / Execution Trace

Initially, all subjects have three possible slots.

Suppose:

A = Slot 1

Since A conflicts with B and C, Slot 1 is removed from their domains.

Therefore:

B = {2, 3}

C = {2, 3}

Now suppose:

B = Slot 2

Since B conflicts with D:

D = {1, 3}

Suppose:

C = Slot 2

Since C conflicts with D and E:

D = {1, 3}

E = {1, 3}

A possible final timetable is:

Subject	**Examination Slot**
Mathematics	Slot 1
Physics	Slot 2
Computer Science	Slot 2
Chemistry	Slot 1
English	Slot 1

All conflicts are satisfied.

11. Complexity Analysis

For n variables and d possible values:

Worst-case Time Complexity:

O(d^n)

Space Complexity:

Approximately O(n), excluding domain-management overhead.

MRV and Forward Checking reduce the practical search space.

12. Advantages and Limitations

Advantages

- Very suitable for scheduling problems.
- Clearly represents variables, domains, and constraints.
- MRV reduces unnecessary choices.
- Forward Checking detects conflicts early.
- Backtracking can recover from incorrect assignments.

Limitations

- Worst-case complexity is exponential.
- Large timetables may contain many constraints.
- Poor variable selection can increase search time.

13. Conclusion

CSP is an effective approach for examination timetable scheduling because it directly represents subjects, time slots, and conflicts.

Using **MRV and Forward Checking** improves efficiency by selecting the most constrained subject and detecting conflicts early.

QUESTION 4 — Minimax and Alpha-Beta Pruning

1. Problem Statement

A game-playing AI needs to determine the best move for the MAX player using the Minimax algorithm.

The game tree contains MAX nodes, MIN nodes, and terminal utility values.

The terminal values are:

3, 5, 2, 9, 1, 4, 6, 7

The objective is to determine the best move for MAX and then apply Alpha-Beta pruning to reduce the number of nodes evaluated.

2. Problem Analysis

Initial State: MAX player at the root.

Goal State: Select the move having the highest guaranteed value.

Assumptions:

- MAX tries to maximize the score.
- MIN tries to minimize the score.
- Both players play optimally.
- Leaf nodes contain utility values.

Constraints:

The game tree must be evaluated according to the Minimax strategy.

3. AI Technique Selection

The selected techniques are:

Minimax Algorithm

and

Alpha-Beta Pruning

Minimax determines the best move assuming both players play optimally.

Alpha-Beta pruning reduces unnecessary node evaluation.

4. Justification of Algorithm

Minimax is suitable for:

- Two-player games.
- Competitive environments.
- Deterministic games.
- Situations where one player's gain is the other player's loss.

Alpha-Beta pruning is suitable because it gives the same result as Minimax but can avoid evaluating branches that cannot affect the final decision.

5. State Space Representation

The game tree is:

MAX

→ Left MIN

→→ MAX: 3, 5

→→ MAX: 2, 9

→ Right MIN

→→ MAX: 1, 4

→→ MAX: 6, 7

6. Variables, Domains, and Constraints

Not Applicable.

This is a game-tree problem, not a CSP.

7. Heuristic Function

Not Applicable.

The terminal nodes already contain utility values, so no heuristic function is required for this example.

8. Algorithm Design / Pseudocode

Minimax

1. Start from the root MAX node.
2. Expand the game tree.
3. At MAX nodes, select the maximum child value.
4. At MIN nodes, select the minimum child value.
5. Continue until the root receives a value.
6. Select the branch with the highest value for MAX.

Alpha-Beta Pruning

1. Start with alpha = negative infinity.
2. Start with beta = positive infinity.
3. Evaluate the game tree from left to right.
4. Update alpha at MAX nodes.
5. Update beta at MIN nodes.
6. If alpha becomes greater than or equal to beta, prune the remaining branch.
7. Continue until the root value is determined.

9. Flowchart / Algorithm Diagram

START

↓

Start at MAX

↓

Evaluate MIN children

↓

Evaluate MAX children

↓

Compare leaf values

↓

Propagate values upward

↓

Calculate root value

↓

Select best move

For Alpha-Beta:

Update α and β

↓

If α ≥ β

↓

Prune remaining branch

10. Working / Execution Trace

Left Subtree

First MAX node:

Maximum of 3 and 5:

max(3, 5) = 5

Second MAX node:

Maximum of 2 and 9:

max(2, 9) = 9

MIN selects the smaller value:

min(5, 9) = 5

Therefore, left subtree = **5**.

Right Subtree

First MAX node:

Maximum of 1 and 4:

max(1, 4) = 4

Second MAX node:

Maximum of 6 and 7:

max(6, 7) = 7

MIN selects:

min(4, 7) = 4

Therefore, right subtree = **4**.

Root MAX

MAX selects:

max(5, 4) = 5

Therefore:

Best Move = Left

Best Value = 5

Alpha-Beta Pruning

After evaluating the left subtree:

Alpha = 5

In the right subtree, the first MAX node gives:

Beta = 4

Now:

Alpha ≥ Beta

5 ≥ 4

Therefore, the remaining nodes with values **6 and 7** do not need to be evaluated.

Comparison

Algorithm	Nodes Evaluated	Nodes Pruned
Minimax	8	0
Alpha-Beta	6	2

11. Complexity Analysis

Let:

- b = branching factor
- d = depth of the game tree

Minimax

Time Complexity:

O(b^d)

Space Complexity:

O(bd) using depth-first implementation.

Alpha-Beta

Worst-case Time Complexity:

O(b^d)

Best-case Time Complexity:

O(b^(d/2))

Thus, Alpha-Beta can significantly reduce the number of nodes evaluated when the ordering is good.

12. Advantages and Limitations

Minimax Advantages

- Gives an optimal decision against an optimal opponent.
- Simple and systematic.
- Suitable for two-player games.

Minimax Limitations

- Evaluates many nodes.
- Becomes expensive for large game trees.

Alpha-Beta Advantages

- Reduces the number of nodes evaluated.
- Produces the same answer as Minimax.
- Allows deeper game-tree searching.

Alpha-Beta Limitations

- Performance depends on node ordering.
- Large game trees can still be expensive.

13. Conclusion

Minimax determines that the MAX player should choose the **left branch**, with a value of:

5

Alpha-Beta pruning produces the same result while avoiding unnecessary evaluation of the nodes **6 and 7**.

Therefore, Alpha-Beta pruning improves the efficiency of Minimax without changing its final decision.

QUESTION 5 — Online Search for Warehouse Robot

1. Problem Statement

A warehouse robot must travel from a starting location to a destination in a dynamic environment. Obstacles such as boxes, vehicles, or people may appear unexpectedly.

The robot must therefore continuously observe the environment and modify its route when required.

2. Problem Analysis

Initial State: S, the robot's starting position.

Goal State: G, the destination.

Initially, the robot may plan:

S → A → B → G

However, if an obstacle appears at B, the robot must find another route.

Assumptions:

- The robot has sensors.
- The environment can change.
- The robot can update its knowledge.
- Alternative paths are available.

Constraints:

- The robot cannot move through obstacles.
- The robot must eventually reach the destination.
- The robot should avoid unnecessary movement.
- The robot must adapt to environmental changes.

3. AI Technique Selection

The most suitable technique is an **Online Search Agent**.

An Online Search Agent makes decisions while interacting with the environment.

The basic process is:

Observe → Decide → Move → Update → Replan

4. Justification of Algorithm

Online Search is suitable because the warehouse environment is dynamic.

For example, the robot may initially plan:

S → A → B → G

But an obstacle may suddenly appear at B.

The robot detects the obstacle, updates its knowledge, and finds another path such as:

S → A → C → G

This makes Online Search more appropriate than a fixed offline search plan.

5. State Space Representation

Initial path:

S → A → B → G

After an obstacle appears:

S → A → X → G

where X represents a blocked location.

The robot searches for another available route:

S → A → C → G

Therefore, the state space changes according to the environment.

6. Variables, Domains, and Constraints

Not Applicable.

This is an Online Search problem rather than a CSP.

7. Heuristic Function

A heuristic can be used to estimate the distance from the current position to the goal.

For a grid-based warehouse, Manhattan distance can be used:

$$h(n) = |x_n - xG| + |y_n - yG|$$

The heuristic helps the robot select a promising direction.

However, the most important feature of Online Search is that the robot can update its plan when new obstacles are discovered.

8. Algorithm Design / Pseudocode

1. Start the robot at S.
2. Observe the current environment.
3. Identify possible next positions.
4. Check whether the planned path is blocked.
5. If the path is blocked, update the internal map.
6. Search for an alternative path.
7. Select the best available movement.
8. Move the robot.
9. Observe the environment again.
10. Repeat the process until the robot reaches G.

9. Flowchart / Algorithm Diagram

START

↓

Robot at S

↓

Observe Environment

↓

Is the path blocked?

→ **Yes:** Update map → Find alternative path

→ **No:** Continue on current path

↓

Move Robot

↓

Observe Environment Again

↓

Has the robot reached G?

→ **Yes:** STOP

→ **No:** Continue searching

10. Working / Execution Trace

Step 1

The robot starts at S.

Initial planned route:

S → A → B → G

Step 2

The robot moves from S to A.

Current position:

A

Step 3

The robot detects an obstacle at B.

Therefore, the original route cannot be used.

The robot updates its map.

Step 4

The robot searches for alternatives.

Possible alternatives include:

A → C → G

or

A → D → G

Suppose A → C → G has the lower estimated cost.

Step 5

The robot follows:

S → A → C → G

The goal is reached successfully.

Main Process

Observe → Detect obstacle → Update map → Replan → Move → Reach goal

11. Complexity Analysis

Let:

- b = branching factor
- d = search depth

Worst-case time complexity for search can be:

O(b^d)

Worst-case space complexity can be:

O(b^d)

The actual performance depends on:

- Number of obstacles.
- Frequency of environmental changes.
- Quality of the heuristic.
- Number of replanning operations.
- Size of the warehouse.

12. Advantages and Limitations

Advantages

- Works in partially unknown environments.
- Adapts to unexpected obstacles.
- Can change the route during execution.
- Suitable for autonomous robots.
- Does not require complete knowledge of the future environment.

Limitations

- Replanning requires additional computation.
- The robot may take a longer route.
- Sensor errors can affect decisions.
- Frequent environmental changes can reduce efficiency.

Comparison with Informed Search

Feature	Informed Search	Online Search
Environment	Mostly known	Partially unknown
Planning	Before execution	During execution
Adaptation	Limited	High

Feature	Informed Search	Online Search
Dynamic obstacles	Difficult	Suitable
Replanning	Usually limited	Continuous
Warehouse robot	Less suitable in highly dynamic environment	Highly suitable

13. Conclusion

An **Online Search Agent** is highly suitable for a warehouse robot operating in a dynamic environment.

The robot continuously observes its surroundings and changes its route when obstacles appear.

The main working principle is:

Observe → Plan → Move → Update → Replan

EVALUATION RUBRICS

Criteria	Weightage (%)	Excellent (90–100%)	Good (75–89%)	Average (60–74%)	Poor (<60%)
Problem Analysis and Understanding	15%	13–15% – Clearly identifies the problem, initial state, goal state, constraints, and expected outcome.	10–12% – Identifies most problem components with minor omissions.	7–9% – Demonstrates partial understanding with some missing elements.	0–6% – Incorrect or incomplete understanding of the problem.
AI Algorithm Selection & Justification	15%	13–15% – Selects the most appropriate AI algorithm/search technique with strong technical justification.	10–12% – Selects an appropriate algorithm with adequate justification.	7–9% – Selects a partially suitable algorithm with limited justification.	0–6% – Incorrect algorithm selection or no justification.
Algorithm Design / Pseudocode	25%	22–25% – Designs a complete, logical, efficient, and error-free algorithm/pseudocode.	17–21% – Algorithm is mostly correct with minor logical errors.	12–16% – Algorithm is partially complete with several logical errors.	0–11% – Algorithm is incomplete, incorrect, or difficult to understand.
Application of AI Concepts (Greedy, A*, CSP, Minimax, Alpha-Beta, etc.)	15%	13–15% – Correctly applies the relevant AI concepts and techniques throughout the solution.	10–12% – Applies AI concepts correctly with minor mistakes.	7–9% – Demonstrates limited application of AI concepts.	0–6% – Unable to apply the appropriate AI concepts or techniques.

Diagram / State Space / Search Tree Representation	10%	9–10% – Accurate, complete, and well-labelled diagram supporting the solution.	7–8% – Diagram is mostly correct with minor omissions.	5–6% – Diagram is partially complete or lacks clarity.	0–4% – Diagram is missing or incorrect.
Complexity Analysis and Solution Efficiency	10%	9–10% – Correctly analyzes time and space complexity and proposes an efficient solution.	7–8% – Complexity analysis is mostly correct with minor errors.	5–6% – Partial complexity analysis or limited discussion of efficiency.	0–4% – No meaningful complexity analysis or highly inefficient solution.
Originality, Critical Thinking, and Presentation	10%	9–10% – Demonstrates excellent originality, logical reasoning, organized presentation, and explores optimized/alternative solutions.	7–8% – Demonstrates good critical thinking with a clear presentation.	5–6% – Shows limited originality and acceptable presentation.	0–4% – Poor presentation with little or no critical thinking.